Computer Vision Walmart

Revisión inteligente de góndolas con YOLOv8 Nano

Francisco Fernández
David Yáñez
David Salvatierra
Francisco Merino

ÍNDICE

01 Abstract
02 Ajustes a la propuesta
03 Metodología Scrum
04 Desarrollo y evidencias
05 Calidad y arquitectura
06 Conclusiones y reflexiones
07 Bibliografía y anexos

Abstract

Proceso tecnico

TECHNICAL PROGRESS

This project develops a computer vision system for intelligent shelf inspection at Walmart:

- Model & Infrastructure: Analyzes image batches using YOLOv8 with bounding boxes.
- Data Workflow: Incorporates the Roboflow v3 dataset (85/15 train-validation split with seed 42), data augmentation via Albumentations, local data operations, and a semi-automated email notification system.
- Model Metrics: The baseline model (best.pt) achieves 72.8% global mAP50, 78.5% precision, and an inference time of 8.7 ms per image.
- Operational Traffic Light: Translates detections into shelf availability levels: Green (100% - 41%), Yellow (40% - 21%), and Red (20% or lower).
- Core Milestone: Integrates artificial intelligence, business logic, database storage, and automated notifications into a fully functional and reproducible prototype.

Intermediate-high technical English

Arquitectura del sistema CVW

Modelo de Vistas 4+1 de Kruchten aplicado al procesamiento de góndolas

1 Ingesta Batch

Recibe imágenes pregrabadas de góndolas y prepara cada lote.

2 Detector YOLOv8n

best.pt

Detecta las clases "product" y "missing" mediante bounding boxes.

3 Calculadora de semáforo

Cuenta detecciones y calcula el porcentaje de stock visible por bandeja o góndola.

Verde: 100%-41%
Amarillo: 40%-21%
Rojo: ≤20%

4 Alertas

Ante estado Rojo (≤20%), envía ubicación exacta en imagen anotada por correo electrónico.

5 Demo Web

Visualización en GitHub Pages de góndolas, métricas y detecciones.

Flujo de procesamiento

Lectura del lote → Inferencia YOLOv8n → product vs missing → % stock → Semáforo → Alerta

Infraestructura y despliegue

Entrenamiento en nube Google Colab + GPU NVIDIA Tesla T4; exportación de best pt.

Ejecución local/servidor Python modular en VS Code + carpeta de ubicaciones.

Servicios externos Servicio SMTP de correo + GitHub Pages.

Escenarios (+1)

lote de imágenes → detección → alerta al caer a ≤20%.

Lógica

ingresa, YOLOv8n, semáforo, notificador y demo web.

Procesos

lectura → inferencia → conteo → stock → clasificación → consulta → alerta.

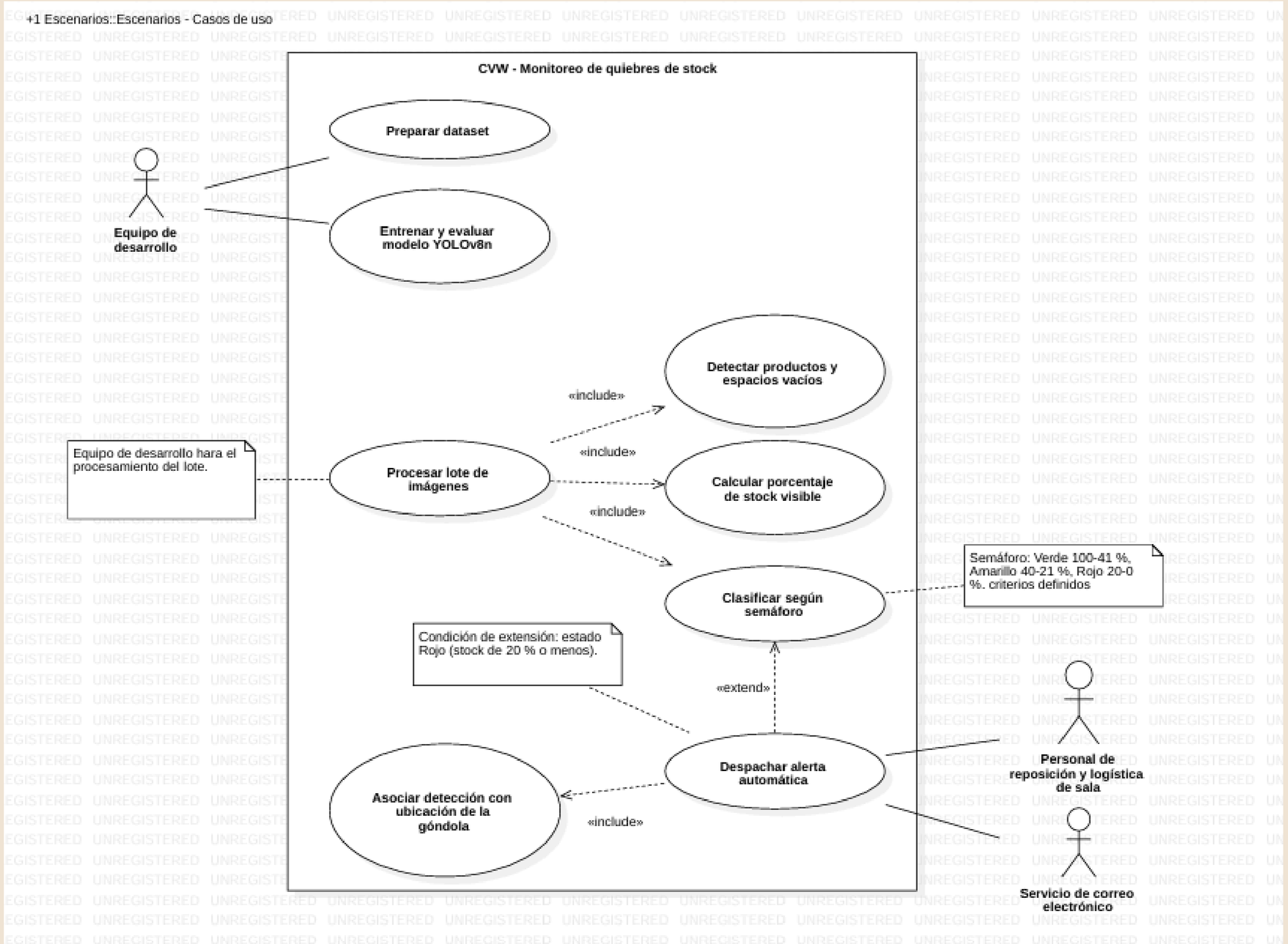
Desarrollo

notebook de Colab
Proyecto: CVW, modelo_v1.ipynb
+ Python modular en VS Code.

Física

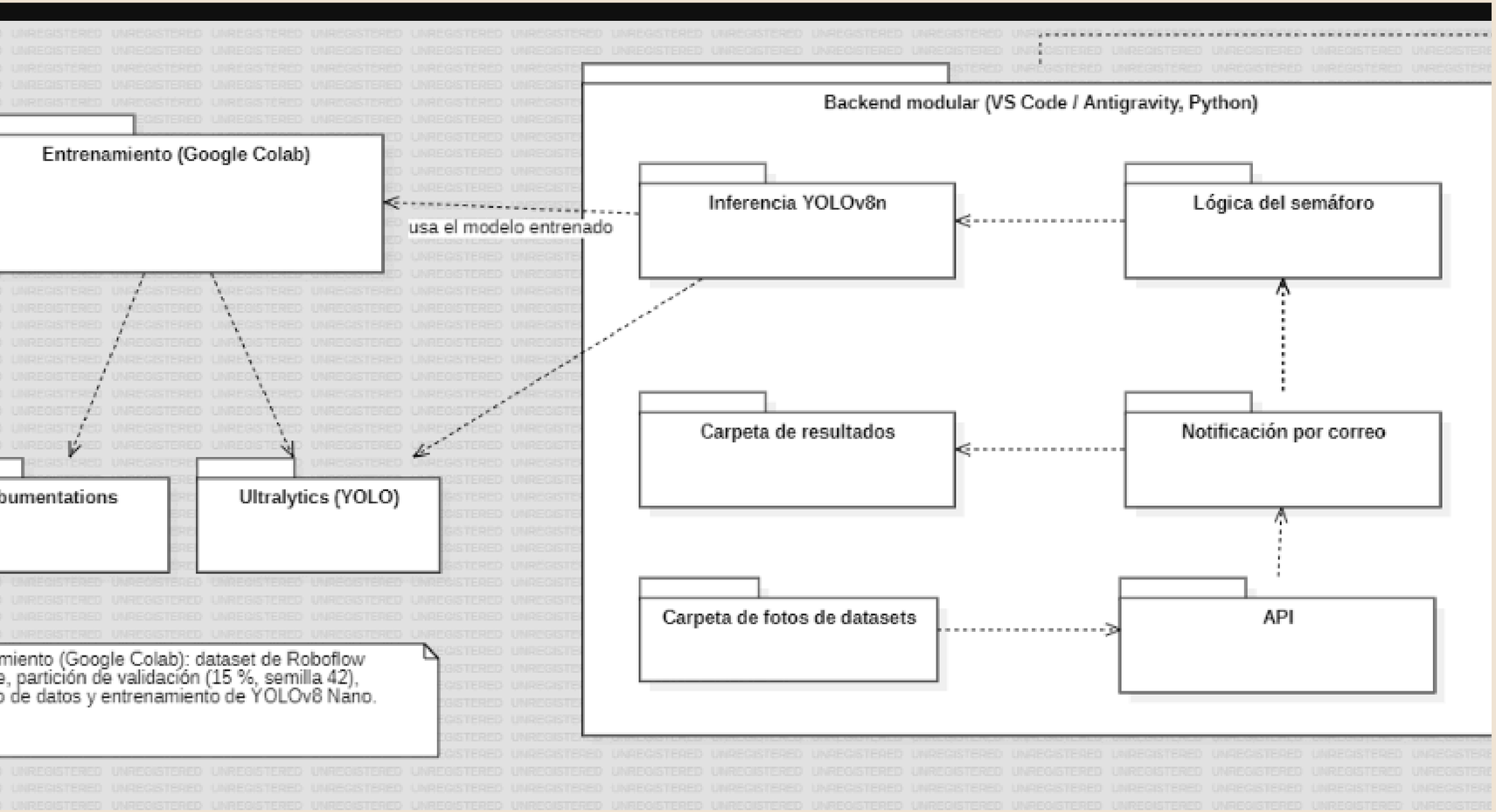
Google Colab, laptop, servicios remotos y GitHub Pages.

vista Casos de uso



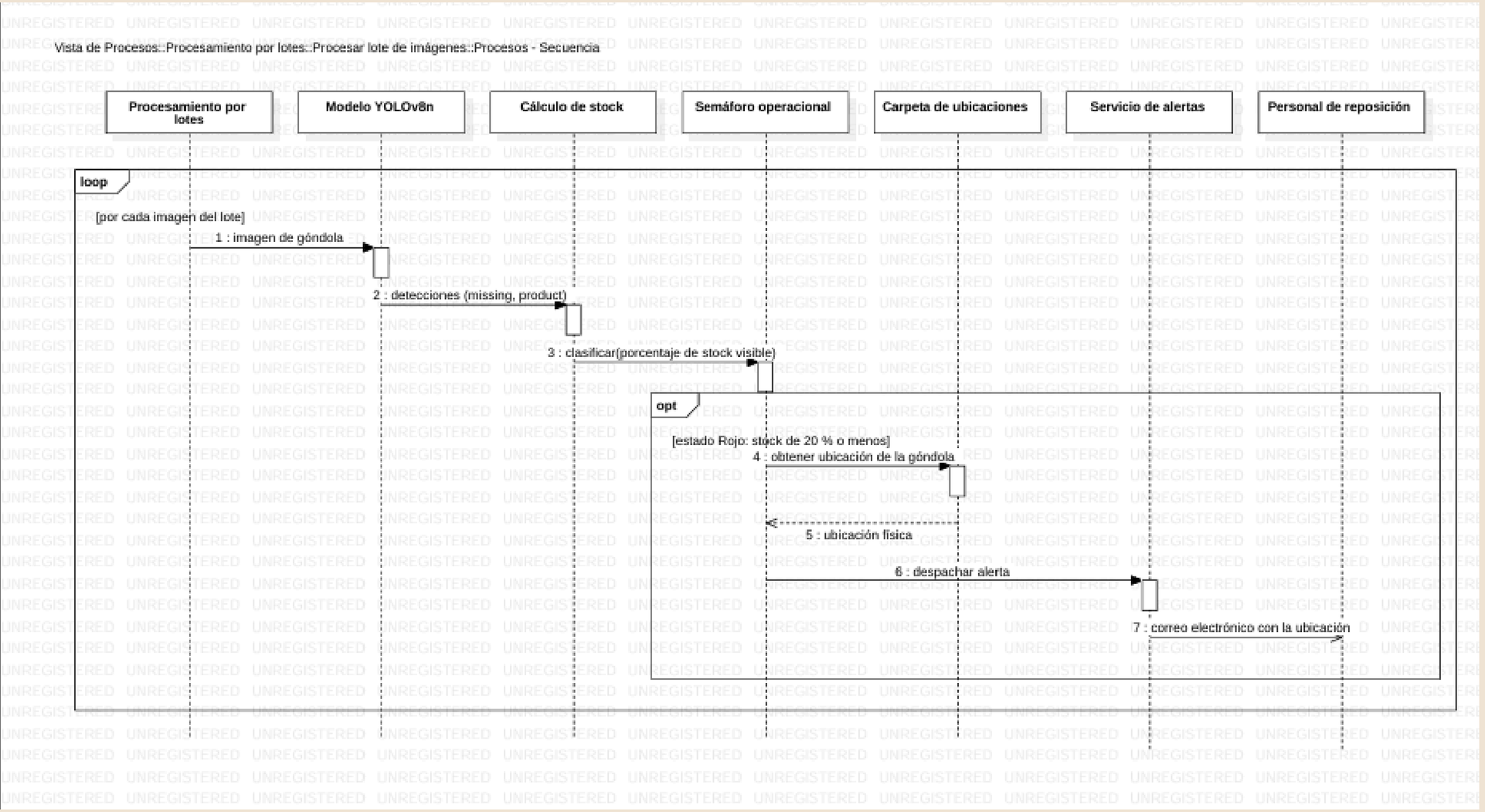
Muestra quién usa el sistema y para qué. El equipo de desarrollo prepara el dataset, entrena el modelo y procesa los lotes de imágenes. Si una góndola queda en Rojo (20 % o menos), se despacha una alerta por correo con su ubicación al personal de reposición.

Vista Desarrollo



Muestra cómo se organiza el código. El entrenamiento se hace en Google Colab con Ultralytics y Albumentations. El backend es modular, en Python, y se divide en inferencia, lógica del semáforo, carpeta de ubicaciones, notificación por correo y API.

Vista De proceso



Muestra el orden en que se procesa cada imagen del lote. Los pasos 1 a 3 ocurren siempre: imagen, detecciones y clasificación. Los pasos 4 a 7 ocurren solo en estado Rojo: ubicación, despacho y correo.

De Fase 1 a Fase 2

Ajustes justificados por dificultades, facilitadores y retroalimentación

ANTES	AHORA
Segmentación de polígonos	**Detección de objetos con YOLOv8 Nano y bounding boxes** Simplifica el etiquetado y mejora la velocidad de inferencia.
Cámaras en vivo	**Procesamiento de imágenes por lotes (batch)** Reduce complejidad, facilita pruebas reproducibles y se ajusta a la disponibilidad real de imágenes.
Semáforo aislado	**Perfeccionamiento del semáforo** Se perfeccionó la lógica del semáforo y se incorporó la notificación vía correo electrónico.

Los cambios responden a dificultades técnicas, facilitadores del proyecto y retroalimentación recibida.

Scrum aplicado al proyecto

Sprints de 2 a 3 semanas con backlog priorizado, seguimiento y entregables incrementales

01 Planificación

02 Desarrollo

03 Revisión

04 Retrospectiva

Resumen de Sprints

SSprint 1 — Adquisición y Preparación de Datos: Descarga del dataset desde Roboflow Universe, partición limpia de validación (15%) con semilla fija (42) y aplicación de aumento de datos (data augmentation) en entrenamiento

- Sprint 2 — Entrenamiento y Prototipado del Modelo IA: Entrenamiento del modelo YOLOv8 Nano en Google Colab con GPU Tesla T4 (100 épocas, batch 16, tamaño 640px, parada anticipada en época 91 con mejor modelo en época 71).

Sprint 3 — Base de Datos y Semáforo Operativo: Creación de desarrollo del script matemático para calcular el porcentaje de presencia de stock y clasificar según las reglas del semáforo, sistema integrado al proyecto con una carpeta de imágenes (BBDD).

Sprint 4 — Modularización en VS Code y Servicio de Alertas: Traslado del código a una estructura modular limpia de backend e integración de sistema de notificación a correo electrónico y creación de api.

Sprint 5 — Pruebas de Integración y Validación de Latencia: Ejecución de pruebas con lotes de imágenes simulando estanterías reales, verificación de latencia de inferencia y ajuste de posibles falsos positivos generados por sombras u oclusiones.

Primeros avances del proyecto

Sprints iniciales de desarrollo

01

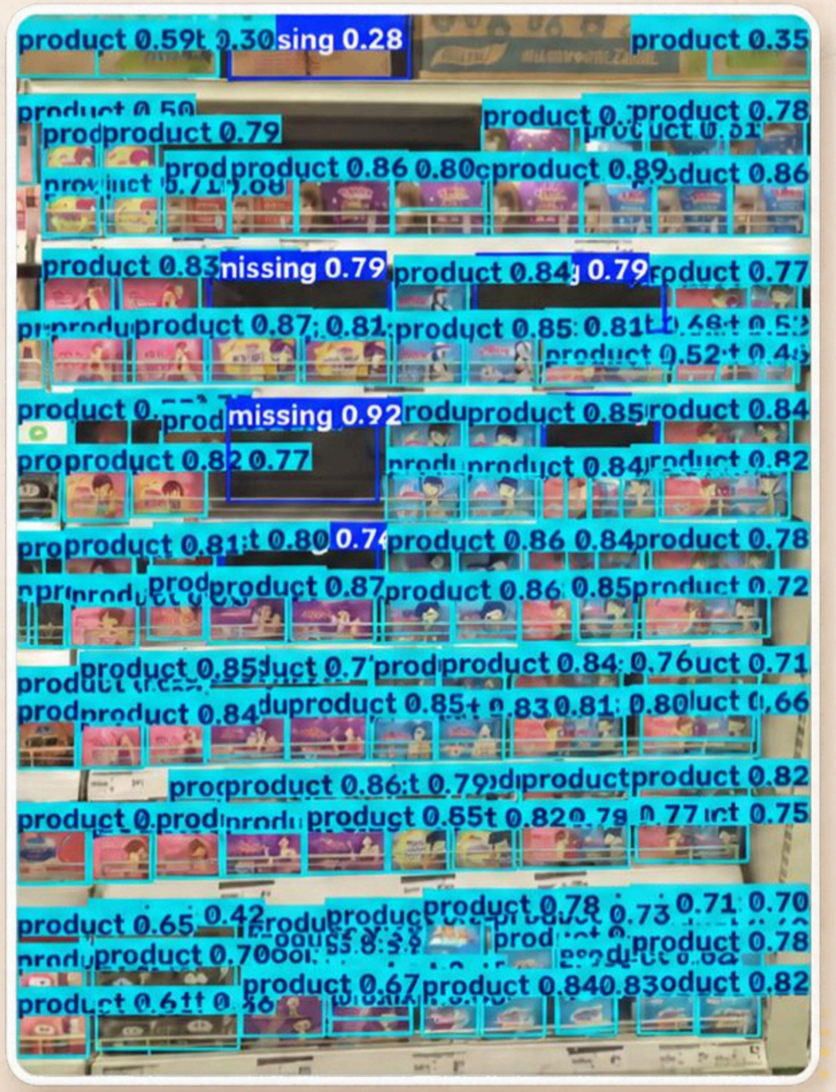
Sprint 1 — Adquisición y Preparación de Datos

Descarga del dataset desde Roboflow Universe, partición limpia de validación (15%) con semilla fija (42) y aplicación de aumento de datos (data augmentation) en entrenamiento.

02

Sprint 2 — Entrenamiento y Prototipado del Modelo IA

Entrenamiento del modelo YOLOv8 Nano en Google Colab con GPU Tesla T4 (100 épocas, batch 16, tamaño 640px, parada anticipada en época 91 con mejor modelo en época 71).



Dataset, preprocesamiento y entrenamiento

Pipeline reproductible ejecutado en Google Colab

DATOS
Roboflow v3

PARTICIÓN
train/valid 85/15
semilla 42

AUMENTACION
Albumentaciones

MODELO
best.pt

MÉTRICAS DEL MEJOR MODELO

mAP50 global
72.8%

Precisión
78.5%

Inferencia
8.7 ms/imagen

Entrenamiento y evaluación ejecutados en Google Colab.

Pipeline: Roboflow dataset (bboxes) -> Aumentación (bboxes) -> Entrenamiento YOLOv8

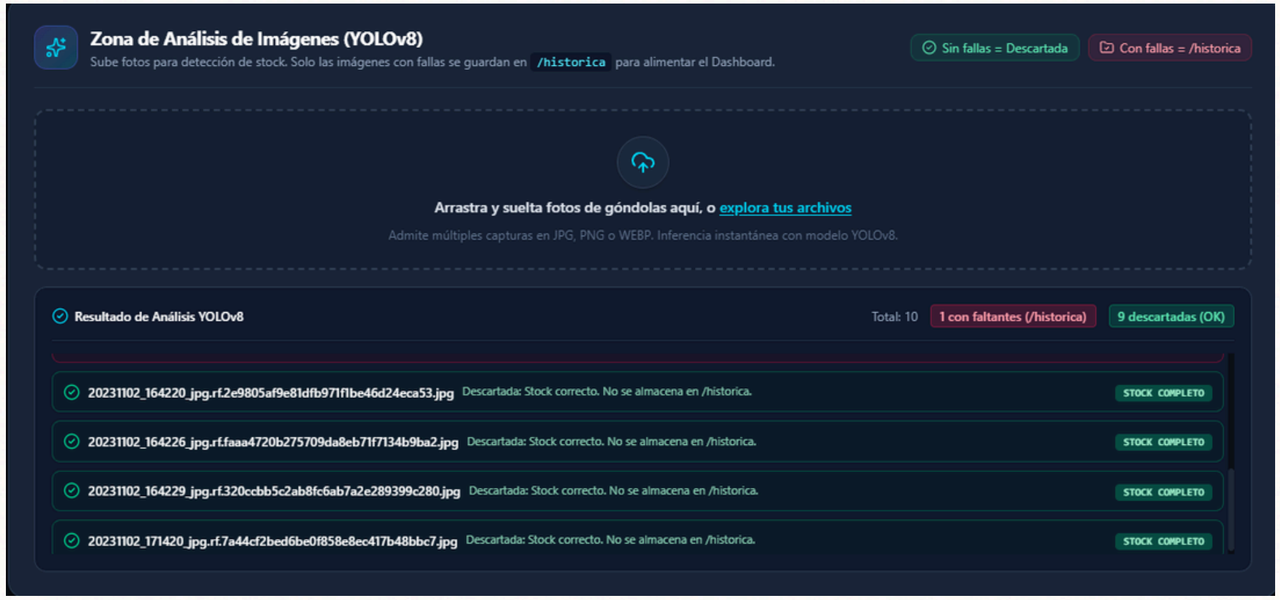
```
!pip install -q ultralytics roboflow albumentations opencv-python-headless
```

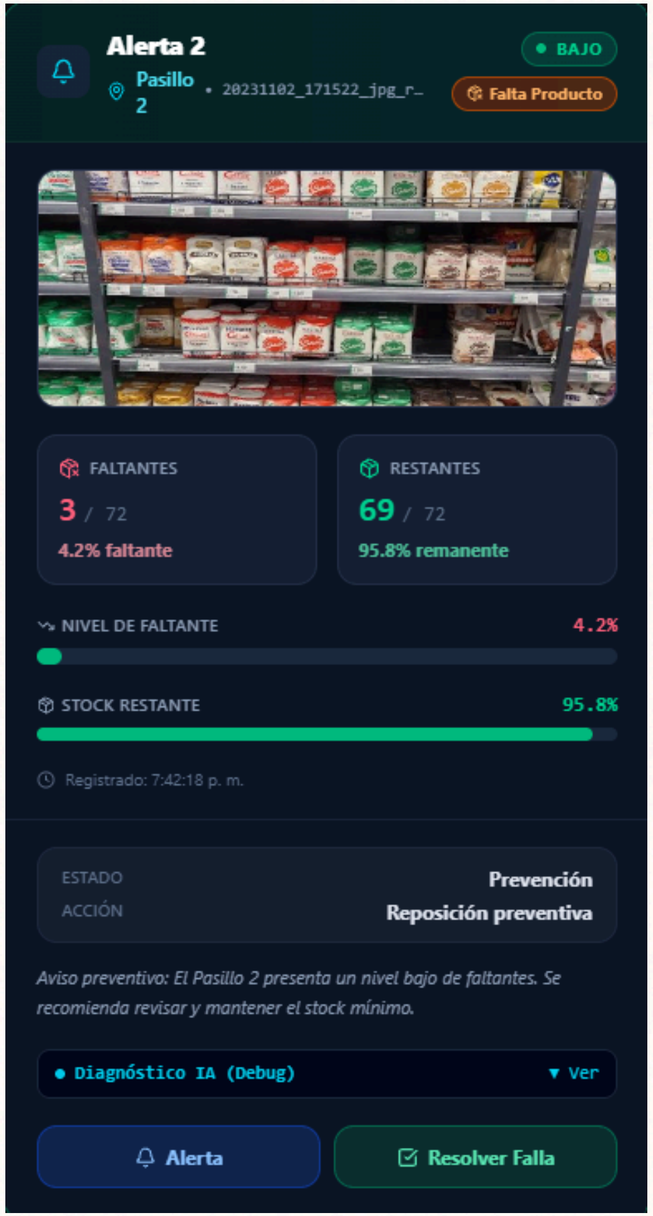
```
import os

ROBOFLOW_API_KEY = "ewDC37UqPQjzs61gnCcK"          # roboflow.com/settings -> API key
RF_WORKSPACE = "final-project-object-detection-for-instore-inventory-management"
RF_PROJECT = "t"
RF_VERSION = 3                                  # usa la versión SIN aumentación de Robof

RAW_DATASET_DIR = "/content/raw_dataset"        # dataset original (bboxes) descargado de
SEG_DATASET_DIR = "/content/seg_dataset"        # dataset con labels de segmentación (pos
AUG_DATASET_DIR = "/content/final_dataset"      # dataset final: seg + aumentado (solo tr

SAM_WEIGHTS = "sam_b.pt"                        # ultralytics descarga el peso automática
N_AUGMENTATIONS_PER_IMAGE = 3                    # copias aumentadas por imagen de train
```





Sprint 3 — Base de Datos y Semáforo Operativo

De la detección de objetos a una clasificación operativa del stock

01

Conteo de detecciones

Procesamiento de las detecciones de las clases **product y missing.**

02

Cálculo del stock visible

Desarrollo del script matemático para calcular el porcentaje de presencia de stock.

03

Clasificación por semáforo

Verde	100%–41%
Amarillo	40%-21%
Rojo	≤20%

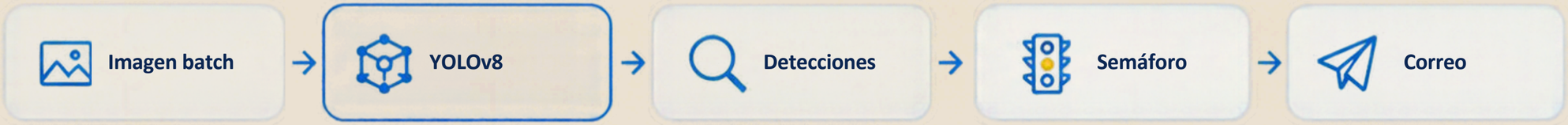
Base de datos del proyecto

Sistema integrado al proyecto mediante una carpeta de imágenes (BBDD) para organizar y utilizar los registros visuales procesados.

El Sprint 3 transforma las detecciones del modelo en una señal operativa interpretable para supervisar el estado de las góndolas.

De la detección a la acción

Arquitectura modular y reproductible



ESTRUCTURA DETECCION

Pasillos

└ **Góndolas**

└ **Niveles**

REGLA DEL SEMÁFORO

Verde: 100% - 41%

Amarillo: 40% - 21%

Rojo: ≤20%

VALIDACIÓN

- Modelo en conjunto valid
- Clasificación del semáforo
- Envío de alertas
- Imágenes anotadas

Conclusiones

Resultados y aprendizajes del proyecto CVW

Detección automatizada

El modelo YOLOv8n permite identificar productos visibles y espacios vacíos en imágenes de góndolas.

Semáforo operativo

El procesamiento de las detecciones permite calcular el porcentaje de stock visible y clasificar el estado de cada góndola.

Alerta para la operación

La integración de la lógica de negocio, la base de datos y la notificación por correo facilita una respuesta oportuna ante niveles críticos de stock.

El proyecto transforma imágenes de góndolas en información operativa para apoyar la supervisión y reposición de productos.

Evidencia

Notebook: Proyecto_CVW_model_v1.ipynb

Notebook de entrenamiento y experimentación del modelo.

Anexo — enlaces de referencia

Repositorio: https://github.com/Linzdhx/Capstone.git

Dataset / Roboflow: https://universe.roboflow.com/final-project-object-detection-for-instore-inventory-management/empty-spaces-in-a-supermarket-hanger-1upsp/fork

Notebook / Colab: https://colab.research.google.com/drive/1XxRRRwO5YSdXfn8StgEmVOEFxJkloeJ4?authuser=2#scrollTo=px79g-7iffkY